🏗️ KIẾN TRÚC HỆ THỐNG - ECOHUB QR SCANNER

📊 SYSTEM OVERVIEW

```
┌──────────────────────────────────────────────────────────────────┐
│                          USER BROWSER                            │
│                       (http://localhost:5000)                    │
└──────────────────┬──────────────────────────────────────────────┘
                   │ HTTP / MJPEG Stream
                   ↓
┌──────────────────────────────────────────────────────────────────┐
│                          FLASK SERVER                            │
│                     (Main Thread: port 5000)                     │
│                                                                  │
│  Routes:  /dashboard  /camera-settings  /storage  /video_feed  │
│  API:     /status  /start_recording  /stop_recording           │
└──────────────┬──────────────────────────────────────────────────┘
               │
               ├──────────────────────────────────────────┐
               │                                          │
               ↓                                          ↓
┌──────────────────────────┐                  ┌──────────────────────────┐
│   CAMERA MANAGER (x2)    │                  │    S3 SERVICE            │
│   (Thread per camera)    │                  │    (Upload Worker)       │
│                          │                  │                          │
│  - Read frames (10ms)    │                  │  - Upload queue (FIFO)   │
│  - Push to AI queue      │                  │  - Multipart upload      │
│  - Push to recorder      │                  │  - Retry mechanism       │
│  - MJPEG encoding        │                  │  - Presigned URLs        │
└──────────┬───────────────┘                  └──────────────────────────┘
           │                                                ↑
           │ Frame Queue (maxsize=2)                        │
           ↓                                                │
┌──────────────────────────┐                                │
│   AI SCANNER (x2)        │                                │
│   (Thread per camera)    │                                │
│                          │                                │
│  - Get frame (0.01s)     │                                │
│  - Resize (480px)        │                                │
│  - Grayscale             │                                │
│  - ZXing decode          │                                │
│  - Cooldown check (5m)   │                                │
│  - Trigger callback      │                                │
└──────────┬───────────────┘                                │
           │
```

```
         │                                                  │
         │ on_code_detected()                              │
         ↓                                                  │
┌──────────────────────────┐                              │
│   VIDEO RECORDER         │                              │
│   (Async Thread)         │                              │
│                          │                              │
│  - Frame queue (90)      │                              │
│  - VideoWriter (15 FPS)  │                              │
│  - Save to videos/       │ ────────────────────────────┘
│  - Queue to upload       │   Upload Task
└──────────────────────────┘
```

🔄 DATA FLOW

1. Camera Frame Flow

```
[USB/RTSP Camera]
        ↓ cv2.VideoCapture()
[Raw Frame (1280x720)]
        ↓ camera_manager.py (_loop)
┌───────┴────────┐
│                │
↓                ↓
[AI Queue]    [Recorder Queue]    [MJPEG Encoder]
(maxsize=2)   (maxsize=90)              ↓
    ↓             ↓              [Browser Stream]
[AI Thread]   [Writer Thread]
    ↓             ↓
[ZXing]       [VideoWriter]
    ↓             ↓
[QR Code]     [videos/*.mp4]
    ↓             ↓
[Callback]    [Upload Queue]
                  ↓
              [S3 Upload]
```

2. QR Detection Flow

```
[Frame from Queue]
        ↓ get_frame_from_queue(timeout=0.01)
[Timestamp Check] → (current_time - last_scan) < 0.2s? → Skip
        ↓ No
[Resize to 480px width]
        ↓
[Convert to Grayscale]
        ↓
```

```
[ZXing Decode]
        ↓
[QR Code Found?] → No → Loop back
        ↓ Yes
[Cooldown Check] → (code in history & < 5m)? → Skip
        ↓ No
[Add to history]
        ↓
[Trigger Callback]
        ↓
[Auto Start Recording]
```

3. Recording Flow

```
[Start Recording Request]
        ↓
[Pause AI Scanner] ← Important!
        ↓
[Create VideoWriter] (15 FPS)
        ↓
[Start Writer Thread]
        ↓
┌──────────────────────────┐
│  Recording Loop          │
│  - Camera pushes frames  │
│  - Recorder queue (90)   │
│  - Writer thread writes  │
│  - Non-blocking          │
└──────────────────────────┘
        ↓
[Stop Recording Request]
        ↓
[Stop Writer Thread]
        ↓
[Clear Frame Queue] ← Fresh frames!
        ↓
[Resume AI Scanner]
        ↓
[Reset AI Scanner]
        ↓
[Queue Upload Task]
```

4. Upload Flow

```
[Video Saved] (videos/*.mp4)
        ↓
[Create UploadTask]
        ↓
[Add to upload_queue] (FIFO)
        ↓
[Upload Worker Thread]
        ↓
```

```
[Get task from queue]
        ↓
[Update status: "uploading"]
        ↓
[File size > 50MB?]
        ↓ Yes              ↓ No
[Multipart Upload]    [Simple Upload]
(10MB chunks)
        ↓                  ↓
[S3 Upload Complete]
        ↓
[Update status: "success"]
        ↓
[Delete local file]
        ↓
[Keep in status_dict] ← Until 00:00 cleanup
```

🧵 THREADING ARCHITECTURE

Thread Hierarchy

```
Main Process (python.exe / EcoHub_QR_Scanner.exe)
│
├── Main Thread (Flask)
│   ├── HTTP Server (werkzeug)
│   ├── Route handlers
│   └── Template rendering
│
├── Camera Thread 1 (USB Camera 0 / RTSP 1)
│   ├── cv2.VideoCapture() read loop
│   ├── Push to _frame_queue (AI)
│   ├── Push to _recorder_queue (if recording)
│   └── MJPEG encode for streaming
│
├── Camera Thread 2 (USB Camera 1 / RTSP 2)
│   └── (same as Camera Thread 1)
│
├── AI Scanner Thread 1
│   ├── Get frames from camera 1 queue
│   ├── Timestamp-based FPS control (5 FPS)
│   ├── ZXing decode
│   ├── Cooldown check
│   └── Trigger callback (main thread)
│
├── AI Scanner Thread 2
│   └── (same as AI Scanner Thread 1)
│
├── Recorder Thread
│   ├── Get frames from _frame_queue
│   ├── Write to cv2.VideoWriter
```

```
│   └── Non-blocking (90-frame buffer)
│
└── Upload Worker Thread
    ├── Process upload_queue (FIFO)
    ├── Multipart S3 upload
    ├── Update upload_status_dict
    └── Cleanup (delete local files)
```

Thread Synchronization

```
# Global Locks
state_lock              # app_state dict
upload_status_lock      # upload_status_dict
camera_status_lock      # camera_status dict

# Queues (Thread-safe)
upload_queue           # Queue.Queue() - FIFO
_frame_queue           # Queue.Queue(maxsize=2) - Camera → AI
_recorder_queue        # Queue.Queue(maxsize=90) - Camera → Recorder

# Thread Events
_stop_event            # threading.Event() - Stop signal
_running               # bool flag - Running state
```

💾 DATA STORAGE

Configuration Storage

```
Development:
    eco_hub_demo/
    ├── config.json         Main config
    ├── config.key          Encryption key (Fernet)
    └── videos/             Local videos

Production (PyInstaller):
    C:\Program Files\EcoHub QR Scanner\
    ├── EcoHub_QR_Scanner.exe
    └── _internal\
        ├── static\         (read-only)
        ├── templates\      (read-only)
        ├── config.json     (default, read-only)
        └── config.key      (default, read-only)

    C:\Users\<user>\EcoHub_QR_Scanner\
    ├── config.json         (writable copy)
```

```
    ├── config.key          (writable copy)
    └── videos\             (writable)
```

Memory Storage

```
# In-Memory State (app.py)
app_state = {
    "current_order_code": None,
    "current_order_info": None,
    "is_recording": False,
    "recording_start": None,
    "recording_order_code": None,
    "auto_record_on_qr": False,
}

upload_status_dict = {
    "filename1.mp4": UploadTask(...),
    "filename2.mp4": UploadTask(...),
}

camera_status = {
    "initialized": False,
    "running": False,
    "error": None,
    "last_test": None,
}

# AI Scanner State (ai_scanner.py)
_code_history = {
    "QR123": timestamp,
    "QR456": timestamp,
}
```

Persistent Storage

```
# Local Files
videos/
├── ORDER123_20260209_150000.mp4
├── ORDER456_20260209_151000.mp4
└── ...

# S3 Storage
s3://<bucket>/
├── ORDER123_20260209_150000.mp4
├── ORDER456_20260209_151000.mp4
└── ...
```

🔐 SECURITY ARCHITECTURE

Encryption Layer

```
[User Input] (S3 Access Key, Secret Key)
        ↓
[ConfigEncryption] (Fernet)
        ↓ encrypt()
[Encrypted String] (base64)
        ↓
[config.json] (s3_config.access_key_encrypted)
        ↓
[Load at runtime]
        ↓
[ConfigEncryption] (Fernet)
        ↓ decrypt()
[Plain Text] (boto3 client)
        ↓
[S3 API Call]
```

Key Management

```
[First Run]
        ↓
[Generate UUID password]
        ↓
[Machine-specific salt] (platform.node() + machine() + system())
        ↓
[PBKDF2-HMAC-SHA256] (100,000 iterations)
        ↓
[32-byte key] → [Base64 encode] → [44 bytes]
        ↓
[Save to config.key]
        ↓
[Fernet(key)] → Encryption/Decryption
```

📡 API ARCHITECTURE

REST Endpoints

```
Public Endpoints (require session):
├── GET  /                      Login page
├── POST /                      Login submit
├── GET  /dashboard             Dashboard (SSR)
```

```
├── GET  /camera-settings       Camera settings (SSR)
├── GET  /storage-settings      S3 settings (SSR)
└── GET  /storage               Storage page (SSR)

API Endpoints (JSON):
├── POST /test_camera           Test camera availability
├── POST /start_cameras         Start camera threads
├── POST /stop_cameras          Stop camera threads
├── POST /start_recording       Start video recording
├── POST /stop_recording        Stop video recording
├── POST /reset_order           Reset current order code
├── GET  /status                Get realtime status (polling)
├── GET  /upload-status         Get upload status (polling)
└── GET  /download/<filename>   Generate presigned URL

Streaming Endpoints:
└── GET  /video_feed/<index>    MJPEG stream
```

Response Format

```
// Success
{
  "success": true,
  "message": "Operation successful",
  "data": { ... }
}

// Error
{
  "success": false,
  "error": "Error message"
}

// Status (polling)
{
  "current_order_code": "ORDER123",
  "order_info": { ... },
  "is_recording": true,
  "recording_seconds": 15
}
```

🎨 FRONTEND ARCHITECTURE

Template Inheritance

```
base.html (Bootstrap 5 + Custom CSS)
├── dashboard.html
```

```
├── camera_settings.html
├── storage_settings.html
├── storage.html
└── login.html
```

JavaScript Architecture

```
// main.js
├── fetchStatus()          // Poll /status every 1s
├── startRecording()       // POST /start_recording
├── stopRecording()        // POST /stop_recording
├── resetOrder()           // POST /reset_order
└── formatSeconds()        // Utility

// storage.html (inline)
├── switchTab()            // Tab switching
├── fetchUploadStatus()    // Poll /upload-status every 2s
└── deleteVideo()          // DELETE /s3/delete

// camera_settings.html (inline)
├── testCamera()           // POST /test_camera
├── startCameras()         // POST /start_cameras
├── stopCameras()          // POST /stop_cameras
└── toggleCamera()         // Enable/disable camera config
```

📦 DEPLOYMENT ARCHITECTURE

Development Environment

```
Developer Machine
├── Python 3.11
├── pip install -r requirements.txt
├── python app.py
└── Access: http://127.0.0.1:5000
```

Production Build

```
Build Machine
├── Python 3.11
├── PyInstaller 6.0.0+
├── pyinstaller build.spec --clean
└── Output: dist/EcoHub_QR_Scanner/

Installer Machine
├── Inno Setup 6.7.0
```

```
├── ISCC.exe setup.iss
└── Output: installer_output/EcoHub_QR_Scanner_Setup_v1.0.0.exe
```

End User Machine

```
Windows 10/11
├── Run installer
├── Install to: C:\Program Files\EcoHub QR Scanner\
├── Data folder: C:\Users\<user>\EcoHub_QR_Scanner\
└── Access: Double-click desktop icon → http://127.0.0.1:5000
```

🔄 STATE MACHINE

Camera State

```
[Not Started] → Test → [Testing]
                  ↓ OK
              [Tested]
                  ↓ Start
              [Running] ←──┐
                  ↓ Stop   │
              [Stopped] ───┘
                  ↓ Restart
              [Running]
```

Recording State

```
[Idle] → Start → [Recording]
                     ↓
                 [Recording]
                     ↓ Stop
                 [Saving]
                     ↓
                 [Uploading]
                     ↓
                 [Success]
                     ↓
                 [Cooldown] (5m)
                     ↓
                 [Idle]
```

Upload State

```
[Local File] → Queue → [Pending]
                          ↓
                      [Uploading] (progress %)
                          ↓
                    [Success/Failed]
                          ↓
                    [Cleanup at 00:00]
```

📊 PERFORMANCE METRICS

Throughput

```
Camera Read:     25-30 FPS (USB), 20-25 FPS (RTSP)
AI Process:      5 FPS (timestamp-based)
Recording:       15 FPS (async writer)
Upload:          10 MB/s chunks (multipart)
```

Latency

```
Camera to AI:    20-50 ms (queue)
QR Detection:    100-200 ms (ZXing)
Start Recording: 200 ms (pause AI + init)
Stop Recording:  100 ms (signal + cleanup)
Upload Init:     < 100 ms (queue)
```

Resource Usage

```
CPU:     15-30% (2-core, recording + AI)
RAM:     200-400 MB (base + buffers)
Disk:    ~50 MB/min (recording at 15 FPS)
Network: Depends on upload speed
```

🔍 MONITORING & LOGGING

Console Logs

```
[STARTUP]      App initialization logs
[CAMERA DEBUG] FPS, queue size, dropped frames
[AI DEBUG]     FPS, timing breakdown, dict size
[UPLOAD WORKER] Upload status, success/failed
[ERROR]        Error messages
[INFO]         Info messages
```

Metrics Collection

```
# Camera Manager
- Frames read per second
- Queue full count (drops)
- Current queue size

# AI Scanner
- Scans per second
- Total processing time
- History dict size

# Upload Worker
- Upload queue size
- Upload success/failed count
- Upload speed (MB/s)
```

Version: 1.0.0
Updated: 09/02/2026
Team: EcoHub Development